

Um Estudo sobre o Contraste e a Comparação entre o Algoritmo de Bellman-Ford e o Algoritmo de Dijkstra

Thippeswamy.K. Prof. Assistente e Chefe de Depto., Dept. de Ciência da Informação e Eng., RLJIT, Kodigehalli, Doddaballapura, Bangalore, Distrito Rural, Karnataka, Índia Email: thippeswamy_yadav@yahoo.co.in

Hanumanthappa.J. Prof. Assistente, Dept. de Estudos em Ciência da Computação, Universidade de Mysore, Manasagangotri, Mysore, Karnataka, Índia Email: hanums_j@yahoo.com

Dr. Manjaiah D.H. Prof. Associado e Presidente, Dept. de Ciência da Computação, Universidade de Mangalore, Mangalagangothri, Mangalore, Karnataka, Índia Email: ylm321@yahoo.co.in

Resumo - Neste artigo, fizemos um estudo sobre os dois conhecidos algoritmos de busca de caminho mais curto, que são usados em roteamento. Eles são o algoritmo de Bellman-Ford e o algoritmo de Dijkstra. Eles foram comparados com base em seu tempo de execução. A análise da comparação é fornecida de forma breve.

1. INTRODUÇÃO

Hoje, uma internet pode ser tão grande que um único protocolo de roteamento não consegue lidar com a tarefa de atualizar a tabela de roteamento de todos os roteadores. Por este motivo, a internet é dividida em sistemas autônomos. O roteamento dentro de um sistema autônomo é chamado de roteamento intradomínio. E a comunicação entre sistemas autônomos é chamada de roteamento interdomínio. O algoritmo de Bellman-Ford e os algoritmos de Dijkstra são dois algoritmos populares usados no roteamento intradomínio para atualizar as tabelas de roteamento.

1.1 Algoritmo de Bellman-Ford

O algoritmo de Bellman-Ford, às vezes referido como Algoritmo de Correção de Rótulos, calcula os caminhos mais curtos de fonte única em um dígrafo ponderado (onde alguns dos pesos das arestas podem ser negativos). Bellman-Ford, em sua estrutura básica, é muito semelhante ao algoritmo de Dijkstra, mas em vez de selecionar de forma gulosa o nó de peso mínimo ainda não processado para relaxar, ele simplesmente relaxa todas as arestas, e faz isso $|V| - 1$ vezes, onde V é o número de vértices no grafo. As repetições permitem que as distâncias mínimas se propaguem com precisão por todo o grafo, uma vez que, na ausência de ciclos negativos, o caminho mais curto só pode visitar cada nó no máximo uma vez. Diferente da abordagem gulosa, que depende de certas premissas estruturais derivadas de pesos positivos, essa abordagem direta se estende ao caso geral.

Procedimento

```
Bellman-Ford (list vertices, list edges, vertex source)
// This implementation takes in a graph, represented as lists of vertices
// and edges, and modifies the vertices so that their distance and
// predecessor attributes store the shortest paths.

// Step 1: Initialize graphe
for each vertex v in vertices:
    if v is source then v.distance := 0
    else v.distance := infinity
    v.predecessor := null

// Step 2: relax edges repeatedly
for i from 1 to size(vertices)-1:
    for each edge uv in edges:
```

```

    u := uv.source
    v := uv.destination
    // uv is the edge from u to v
    if v.distance > u.distance + uv.weight:
        v.distance := u.distance + uv.weight
        v.predecessor := u

// Step 3: check for negative-weight cycles
for each edge uv in edges:
    u := uv.source
    v := uv.destination
    If v.distance > u.distance + uv.weight:
        error "Graph contains a negative-weight cycle"

```

1.2 Algoritmo de Dijkstra

O algoritmo de Dijkstra, concebido pelo cientista da computação holandês Edsger Dijkstra em 1959, é um algoritmo de busca em grafos que resolve o problema do caminho mais curto de fonte única para um grafo com custos de caminho de aresta não negativos, produzindo uma árvore de caminho mais curto. Este algoritmo é frequentemente usado em roteamento. Para um determinado vértice fonte (nó) no grafo, o algoritmo encontra o caminho com o menor custo (ou seja, o caminho mais curto) entre esse vértice e todos os outros vértices. Ele também pode ser usado para encontrar custos de caminhos mais curtos de um único vértice para um único vértice de destino, parando o algoritmo assim que o caminho mais curto para o vértice de destino for determinado.

Procedimento

Deve-se notar que a distância entre os nós também pode ser referida como peso.

1. Crie uma lista de distâncias, uma lista de vértices anteriores, uma lista de visitados e um vértice atual.
2. Todos os valores na lista de distâncias são definidos como infinito, exceto o vértice inicial, que é definido como zero.
3. Todos os valores na lista de visitados são definidos como falso.
4. Todos os valores na lista anterior são definidos com um valor especial significando que são indefinidos, como nulo.
5. O vértice atual é definido como o vértice inicial.
6. Marque o vértice atual como visitado.
7. Atualize as listas de distância e anteriores com base nos vértices que podem ser alcançados imediatamente a partir do vértice atual.
8. Atualize o vértice atual para o vértice não visitado que pode ser alcançado pelo caminho mais curto a partir do vértice inicial.
9. Repita (a partir do passo 6) até que todos os nós sejam visitados.

```

{
    int *distance; // Should be allocated
    int i, j;

    if (distance == NULL) Then
    {
        fprintf(stderr, "malloc() failed\n");
        exit (EXIT_FAILURE);
    }
    for (i = 0; i < nodecount; ++i)

```

```

        distance[i] = INFINITY;
    distance[source] = 0;

    for (i = 0; i < nodecount; ++i)
    {
        for (j = 0; j < edgecount; ++j)
        {
            if (distance[edges[j].source] != INFINITY)
            {
                int new_distance =
                    distance[edges[j].source] + edges[j].weight;
                if (new_distance < distance[edges[j].dest])
                    distance[edges[j].dest] = new_distance;
            }
        }
    }
    for (i = 0; i < edgecount; ++i)
    {
        If (distance[edges[i].dest] >
            distance[edges[i].source] + edges[i].weight) {
            puts ("Negative edge weight cycles detected!");
            free (distance);
            return;
        }
    }
}

```

2. ALGORITMOS

Os algoritmos para os dois procedimentos acima são fornecidos a seguir.

2.1 Algoritmo de Bellman-Ford

Complexidade de tempo Entrada: Aresta arestas[], int contagem_arestas, int contagem_nos, int origem Saída: Tabela de roteamento n - Contagem de nós m - Contagem de arestas Tempo $\propto 10 \cdot n + 18 \cdot m + 27 \cdot n \cdot m + 3$ i.e. Tempo $\propto n \cdot m$ Tempo $\propto 27 \cdot \text{Arestas} \cdot \text{Nós}$ Tempo $\propto (\text{número de nós}) \cdot (\text{número de arestas})$

Para um caso geral em que o número de arestas é igual ao número de nós, podemos escrever ($m = n$): Tempo $\propto 27 \cdot n^2 + 28 \cdot n + 3$ i.e. Tempo $\propto 27n^2$

2.2 Algoritmo de Dijkstra

```

1 function Dijkstra (Graph, source):
2   for each vertex v in Graph:           // Initializations
3     dist[v] := infinity                 // Unknown distance from source to v
4     previous[v] := undefined            // Previous node in optimal path from source
5   dist[source] := 0                     // Distance from source to source
6   Q := the set of all nodes in Graph    // All nodes in the graph are unoptimized -
7   while Q is not empty:                 // The main loop
8     u := node in Q with smallest dist
9     remove u from Q
10    for each neighbor v of u:           // where v has not yet been removed from Q.
11      alt := dist[u] + dist_between (u, v)
12      if alt < dist[v]                   // Relax (u,v)
13        dist[v] := alt
14        previous[v] := u
15  return previous[]

```

Se estivermos interessados apenas no caminho mais curto entre os vértices *origem* e *destino*, podemos terminar a busca na linha 10 se $u = \text{destino}$. Agora podemos ler o caminho mais curto da origem ao destino por iteração:

```

1 S := empty sequence
2 u := target
3 while defined previous[u]
4     insert u at the beginning of S
5     u := previous[u]
```

Complexidade de tempo O tempo de execução do algoritmo de Dijkstra em um grafo com arestas E e vértices V pode ser expresso como uma função de Arestas e Vértices usando a notação Big-O.

A implementação mais simples do algoritmo de Dijkstra armazena os vértices do conjunto Q em uma lista vinculada ordinária ou array, e a operação Extrair-Min(Q) é simplesmente uma busca linear através de todos os vértices em Q . Neste caso, o tempo de execução é $O(|V|^2 + |E|) = O(|V|^2)$. $O(\text{Vértices}^2 + \text{Arestas})$

Para um caso geral em que o número de arestas é igual ao número de nós, podemos escrever ($m = n$): Tempo $\propto 2n^2$ (a partir do algoritmo acima)

Para um caso de $n = m$, podemos traçar um gráfico (Número de Nós vs. tempo): A partir da análise dos dados, fica claro que, embora a natureza das duas curvas seja a mesma, ou seja, $O(n^2)$, o algoritmo de Bellman Ford requer mais tempo do que o algoritmo de Dijkstra.

A funcionalidade do algoritmo original de Dijkstra pode ser estendida com uma variedade de modificações. Por exemplo, às vezes é desejável apresentar soluções que são menos que matematicamente ótimas. Para obter uma lista classificada de soluções menos que ótimas, a solução ótima é calculada primeiro. Uma única aresta que aparece na solução ótima é removida do grafo, e a solução ótima para este novo grafo é calculada. Cada aresta da solução original é suprimida sucessivamente e um novo caminho mais curto é calculado. As soluções secundárias são então classificadas e apresentadas após a primeira solução ótima.

Diferente do algoritmo de Dijkstra, o algoritmo de Bellman-Ford pode ser usado em grafos com pesos de aresta negativos, desde que o grafo não contenha nenhum ciclo negativo acessível a partir do vértice de origem s . (A presença de tais ciclos significa que não há caminho mais curto, já que o peso total se torna menor a cada vez que o ciclo é atravessado). No entanto, o algoritmo de Bellman-Ford tem outra desvantagem. O algoritmo de Bellman-Ford não impede que loops de roteamento ocorram e sofre do problema de contagem ao infinito. O cerne do problema da contagem ao infinito é que se A disser a B que ele tem um caminho em algum lugar, não há como B saber se ele está no caminho. Para ver o problema claramente, imagine uma sub-rede conectada como A-B-C-D-E-F, e deixe a métrica entre os roteadores ser "número de saltos". Agora suponha que A caia. No processo de atualização de vetor, B percebe que sua rota que outrora era muito curta de 1 para A caiu; B não recebe a atualização de vetor de A. O problema é que B também recebe uma atualização de C, e C ainda não está ciente do fato de que A caiu, então ele diz a B que A está a apenas dois saltos dele, o que é falso. Isso se propaga lentamente pela rede até atingir o infinito (caso em que o algoritmo se corrige, devido à "Propriedade de Relaxamento" do Bellman Ford).

3. CONCLUSÃO

Como mostra a análise, o algoritmo de Bellman-Ford resolve um problema com uma complexidade

de $27n^2$, mas o algoritmo de Dijkstra resolve o mesmo problema com um tempo de execução menor, mas exige que os pesos das arestas sejam não negativos. Portanto, Bellman-Ford é geralmente usado apenas quando há pesos de aresta negativos.

Ambas essas funções resolvem o problema do caminho mais curto de fonte única. A diferença principal no funcionamento dos dois algoritmos é que o algoritmo de Dijkstra não consegue lidar com pesos de aresta negativos. O algoritmo de Bellman-Ford pode lidar com algumas arestas de peso negativo. Deve ser lembrado, no entanto, que se houver um ciclo negativo, não há caminho mais curto.

4. REFERÊNCIAS

[1] en.wikipedia.org/ [2] A note on two problems in connexion with graphs. In *Numerische Mathematik*, 1 (1959), S. 269-271. [3] Thomas H. Cormen, Charles E. Leiserson, Ronald Rivest and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill. 2001. ISBN 0-262-03293-7. Section 24.3: Dijkstra's algorithm, pp.595-601. [4] Robert Sedgewick. *Algorithms in Java*. Third Edition. ISBN 0-201-36121-3. Section 21.7: Negative Edge Weights. <http://safari.oreilly.com/0201361213/ch21lev1sec7> [5] Jin Y. Yen. "An algorithm for Finding Shortest Routes from all Source Nodes to a Given Destination in General Network", *Quart. Appl. Math.*, 27, 1970, 526-530. [6] Richard Bellman: On a Routing Problem, in *Quarterly of Applied Mathematics*, 16(1), pp.87-90, 1958. [7] Lestor R. Ford jr., D. R. Fulkerson: *Flows in Networks*, Princeton University Press, 1962. [8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Section 24.1: The Bellman-Ford algorithm, pp.588-592. Problem 24-1, pp.614-615.

5. AGRADECIMENTOS

Gostaria de agradecer ao Departamento de Ciência e Engenharia da Informação, Faculdade de Engenharia HKBK. Departamento de Estudos em Ciência da Computação, Universidade de Mysore, Manasagangothri & Departamento de Pós-Graduação em Ciência da Computação, Universidade de Mangalore, Mangalore, por fornecer apoio para conduzir este trabalho de pesquisa.

Mr. Hanumanthappa.J. é Professor no DoS em CS, Universidade de Mysore, Manasagangothri, Mysore-06 e atualmente faz Ph.D em Ciência da Computação e Engenharia, da Universidade de Mangalore sob a supervisão do Dr. Manjaih.D.H com o título "IPv6 and Multimedia Stuffs". Seus interesses de ensino e pesquisa incluem Redes de Computadores, Redes Sem Fio e de Sensores, Redes Móveis Ad-Hoc, Sistema de detecção de Intrusão, Segurança de Rede e Criptografia, Protocolos de Internet, Computação Móvel e Cliente Servidor, Gerenciamento de tráfego, Qualidade de Serviço, RFID, Bluetooth, internos do Unix, interno do Linux, Programação de Kernel, Análise e Design Orientado a Objetos etc. Seu foco de pesquisa mais recente é nas áreas de Protocolos de Internet e suas aplicações. Ele recebeu seu diploma de Bacharel em Engenharia em Ciência da Computação e Engenharia da Universidade B.D.T College of Engineering, Davanagere, Karnataka(S), Índia(C), Universidade Kuvempu, Shimoga no ano de 1998 e Mestrado em Tecnologia em CS&Engenharia pelo NITK Surathkal, Karnataka(S), Índia (C) no ano de 2003. Ele tem sido associado como corpo docente do Departamento de Estudos em Ciência da Computação desde 2004. Ele trabalhou como professor na SIR.M.V.I.T.Y.D.L.T.S.V.LT. de Bangalore. Ele orientou cerca de 50 teses de Projeto de BE, B.Tech, M.Tech, MCA, MSc/MS. Ele publicou cerca de 10 artigos técnicos em conferências internacionais e nacionais revisadas por pares. Ele é um membro vitalício da CSI, ISTE, AMIE, IAENG, grupo de rede integrada do TIFAC-CORE em Engenharia de Redes ACM, Associação de Professores de Ciência da Computação (CSTA). Ele também é membro do BOE de todas as universidades de Karnataka, Índia. Ele também visitou a República da China como Professor Visitante para ensinar disciplinas de Ciência da Computação, como SO e Software de Sistema e Engenharia de Software para alunos B.Tech da Universidade Huang Huai no ano de 2008. Ele também visitou a Tailândia como Turista para visitar muitos lugares na Tailândia.

Mr. B.L.D Kumar é Professor Assistente na Faculdade de Engenharia HKBK em Ciência da Informação, Bangalore-45. Seus interesses de ensino e pesquisa incluem Algoritmos, Estrutura de dados, Microprocessador, Redes de Computadores, internos do Unix, interno do Linux, Programação de Kernel, Análise e Design Orientado a Objetos etc. Ele recebeu seu diploma de Bacharel em Engenharia em Ciência da Computação e Engenharia do Shree Siddhartha Institute of Engineering, Tumkur, Karnataka(S), Índia(C), Universidade de Bangalore, Bangalore no ano de 2000 e Mestrado em Tecnologia em Ciência da Computação & Engenharia pela Universidade BDT College of Engineering, Davanagere, Universidade Kuvempu, Shimoga, Karnataka(S), Índia (C) no ano de 2005. Ele tem sido associado como corpo docente do Departamento de Ciência da Informação desde agosto de 2007. Em seus 7 anos de experiência de ensino, ele orientou muitas teses de projeto para BE, B.Tech e se envolveu em atividades técnicas.

Dr. Manjaiah D.H. é atualmente Leitor e Presidente da BoS, tanto na Graduação/Pós-Graduação em Ciência da Computação no Dept. de Ciência da Computação, Universidade de Mangalore, Mangalore. Ele também é membro da BoE de todas as Universidades de Karnataka e outras universidades renomadas na Índia. Ele recebeu o título de doutor pela Universidade de Mangalore, M.Tech. do NITK, Surathkal e B.E., da Universidade de Mysore. O Dr. Manjaiah D.H tem extensa experiência acadêmica, industrial e em pesquisa. Trabalhou em muitos órgãos técnicos, como IAENG, WASET, ISOC, CSI, ISTE e ACS. Ele é autor de mais de 25 artigos de pesquisa em conferências internacionais e periódicos conceituados. Ele é o destinatário de diversas palestras sobre sua área de interesse em muitas ocasiões públicas. Ele é membro de comitês de especialistas de uma AICTE e de vários órgãos técnicos. Ele escreveu um livro-texto em Kannada, intitulado "COMPUTER PARICHAYA", para o benefício de todos os professores e comunidade estudantil de Karnataka. As áreas de interesse do Dr. Manjaiah.D.H são Redes de Computadores e Redes de Sensores, Comunicação Móvel, Pesquisa Operacional, Comércio Eletrônico, Tecnologia da Internet e Programação Web.